LabVIEW – grafické programovanie a dataflow

Programovanie meracích systémov

Ján Šaliga · 2026

Vytvorené s podporou AI

Prečo ďalší programovací nástroj?

- Poznáme základy C, programovali sme Arduino a používame MATLAB, niektorí možno máte skúsenosť s Pythonom. **Prečo ešte LabVIEW?**
- Rôzne prostredia – rôzne typické aplikačné ťažiská

C/Python	Všeobecné programovanie	Všeobecné softvérové aplikácie
MATLAB	Výpočty, dáta	Analýza a spracovanie
LabVIEW	Merací HW, dáta, používateľ	Meracia aplikácia

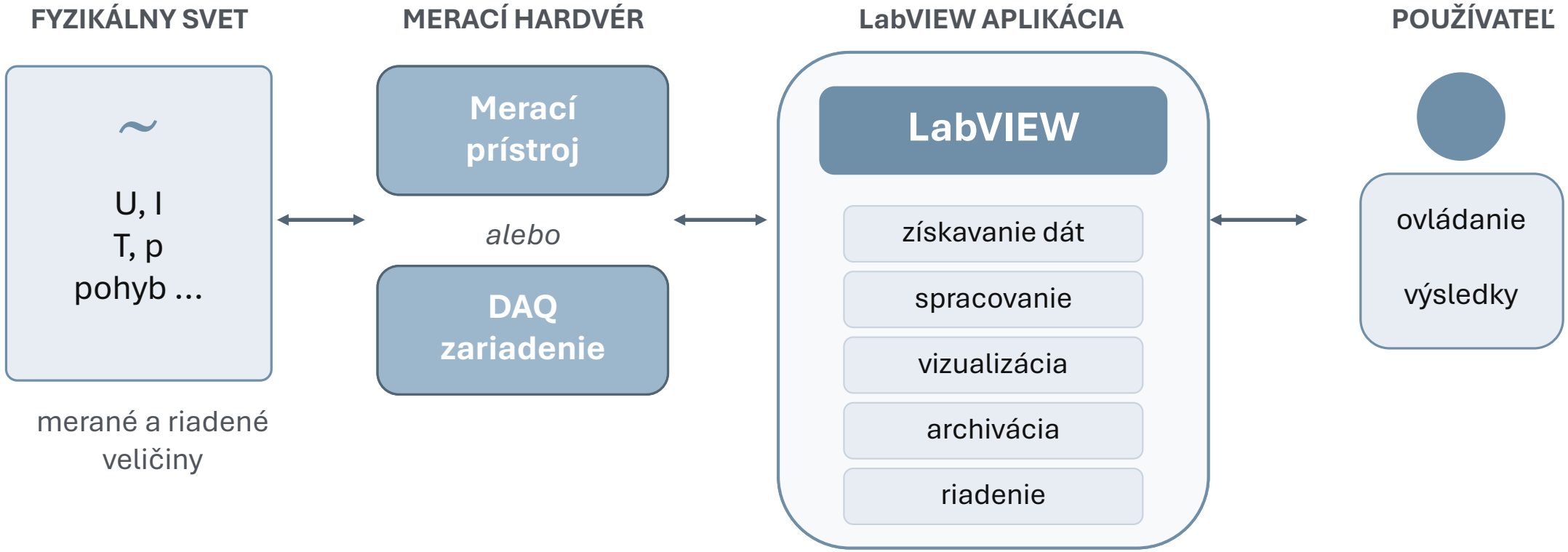
LabVIEW je prostredie špecializované na tvorbu meracích a riadiacich aplikácií.

Programovanie + merací hardvér + získavanie dát + spracovanie + vizualizácia



LabVIEW v meracom systéme

- Meracia aplikácia prepája fyzikálny svet, merací hardvér a používateľa.



Úlohou programu nie je iba vypočítať výsledok, ale riadiť priebeh merania a tok dát.

Viaceré činnosti môžu prebiehať súčasne.

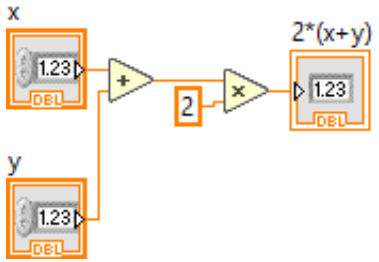
LabVIEW nie je iba „kreslenie programu“

- Grafický zápis je viditeľný rozdiel, ale podstatný rozdiel je **model vykonávania programu**.

Textový program
C/MATLAB/Python

```
x = ...
y = ...
z = x + y
result = 2*z
```

LabVIEW
graf ikon prepojených dátovými spojmi (wires)



Rovnaký algoritmus môžeme zapísať rôznymi spôsobmi.

- Je teda LabVIEW iba grafický zápis toho istého programu?
- NIE – rozdiel je hlbší.

LabVIEW používa model dataflow – vykonávanie programu je riadené dostupnosťou dát na požadovaných vstupoch jednotlivých ikon.

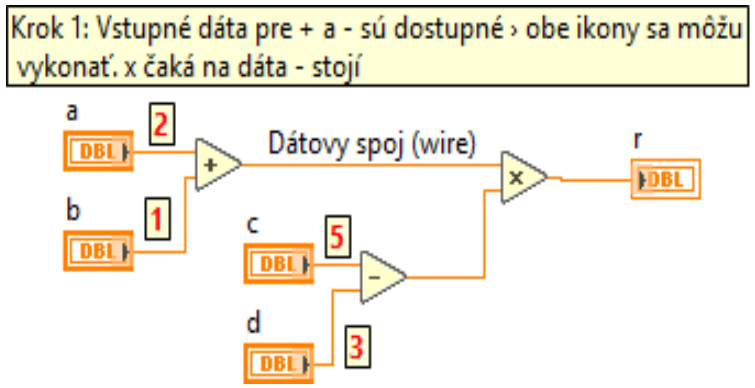


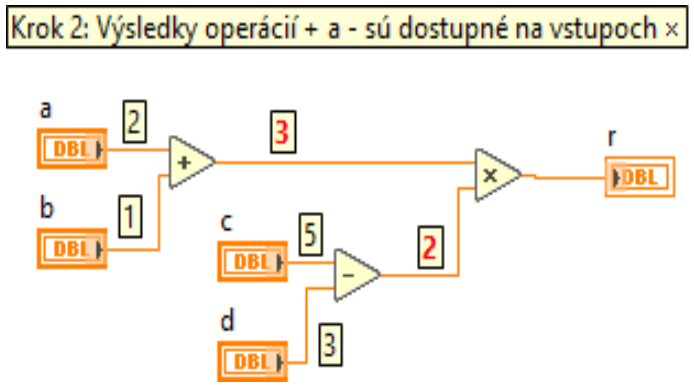
Dataflow – kedy sa ikona vykoná?

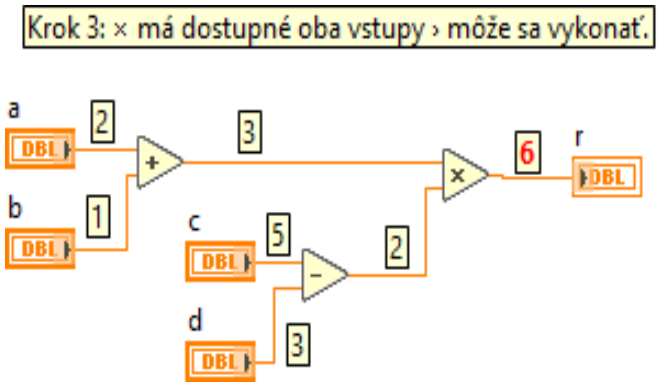
- Jeden jednoduchý príklad: r=(a+b)(c−d).
- Priebeh vykonávania:

Základné pravidlo dataflow:
Ikona sa môže vykonať, keď sú dostupné dáta na všetkých jej požadovaných vstupoch.

vstup (input) → ikona → výstup (output)





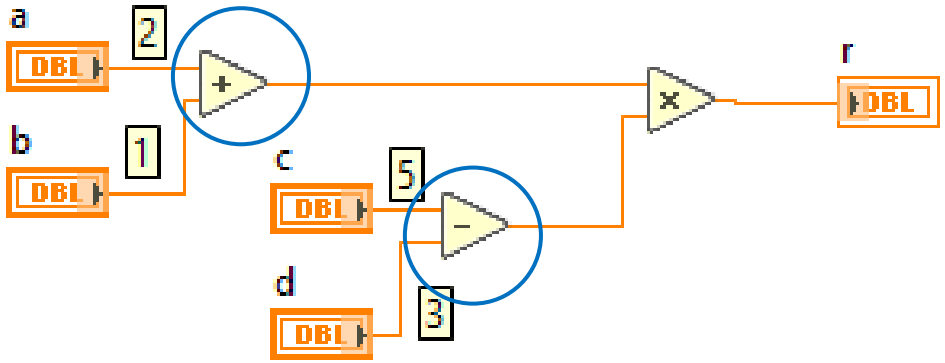


Poradie vykonávania ikon neurčuje ich poloha v Block Diagram, ale dátové závislosti medzi nimi.



Dataflow a paralelné vykonávanie

- Ktorá operácia sa musí vykonať ako prvá – + alebo –?



+ nepotrebuje výsledok –
– nepotrebuje výsledok +

↓

Medzi operáciami nie je dátová závislosť.

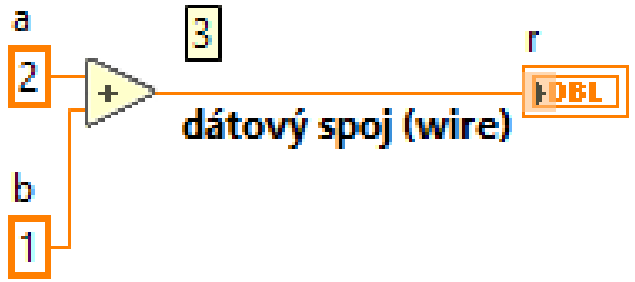
- Dataflow neurčuje, ktorá z nich sa vykoná ako prvá.
- Časti programu, medzi ktorými nie je dátová závislosť, sa **môžu** vykonávať paralelne (parallel execution).

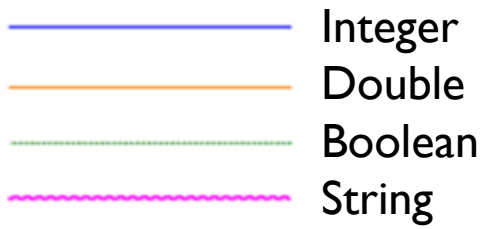
Dataflow umožňuje paralelné vykonávanie nezávislých častí programu bez explicitného určovania ich poradia.
Paralelizmus prirodzene vyplýva z modelu dataflow.



Dátový spoj (wire)

- Wire nie je iba grafická čiara medzi ikonami.
- Wire prenáša dáta z výstupu jednej ikony na vstup ďalšej.
- Má tri významy:
  - **Prenáša hodnotu** (data), hodnota vytvorená na výstupe je dostupná na pripojených vstupoch.
    [+] —— 3.0 ——▶ [...]
  - **Určuje dátový typ** (data type).
    Farba a vzhľad wire súvisia s typom prenášaných dát.
  - **Vytvára dátovú závislosť** (data dependency)
    [A] ——————▶ [B],
    B nemôže pracovať s výsledkom A, kým tento výsledok nie je dostupný.





Wire prenáša dáta, reprezentuje ich dátový typ a vytvára dátovú závislosť medzi časťami programu.

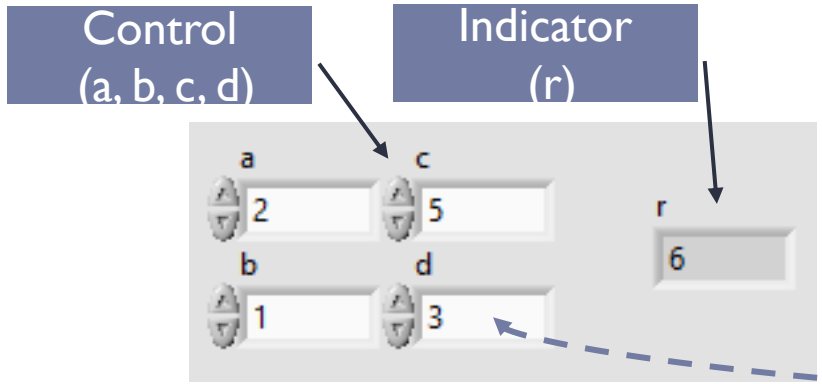


VI: Front Panel a Block Diagram

VI = Virtual Instrument
súbor *.vi

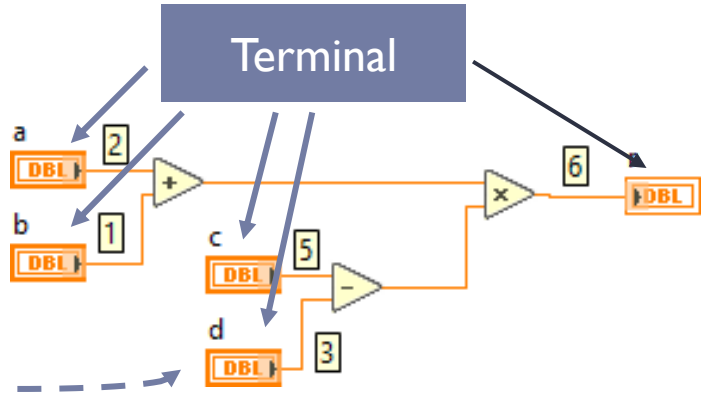
Každý prvok na Front Paneli má svoj terminál (terminal) v Block Diagrame.

- **Front Panel**
  používateľské rozhranie VI



- **Control**
  vstup od používateľa / zdroj dát pre program
- **Indicator**
  výstup programu / zobrazenie dát používateľovi

- **Block Diagram**
  grafický zdrojový kód VI



- **Terminal**
  Terminal – reprezentácia prvku z Front Panel v Block Diagram

Front Panel zabezpečuje interakciu s používateľom, Block Diagram obsahuje program a určuje spracovanie a tok dát.



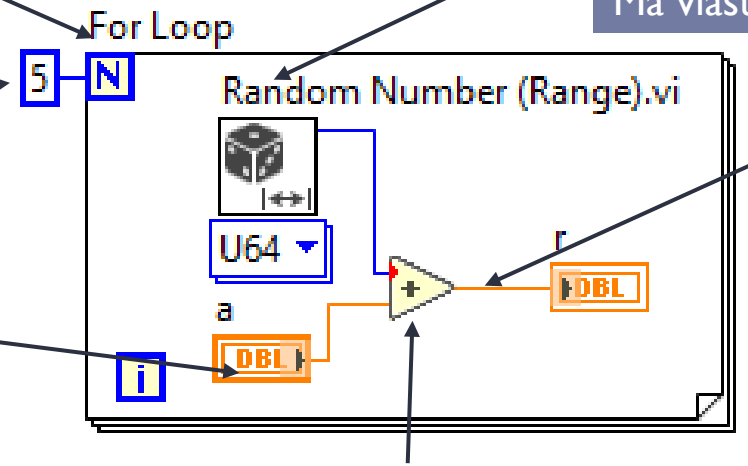
Základné prvky Block Diagramu

- Block Diagram je vytvorený z grafických objektov prepojených dátovými spojmi (wires).

Structure
riadi vykonávanie časti diagramu
Napr. For Loop.

SubVI
VI použité ako podprogram v inom VI
Má vlastný Block Diagram

Constant
konštantná hodnota
definovaná v Block Diagrame
Napr. numerická 5.



Wire
prenáša dáta medzi terminálmi

Terminal
miesto vstupu alebo výstupu dát,
Napr. väzba na prvok vo Front Paneli

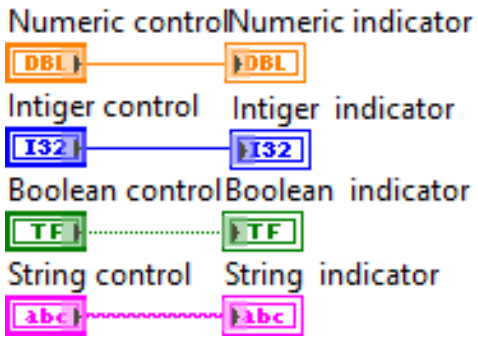
Function
žltá základná ikona
implementovaná funkcia
vs.
SubVI
farebná ikona
VI použité ako podprogram

Function (Function ≠ SubVI)
základná operácia/funkcia LabVIEW

Pri čítaní Block Diagram sledujeme:
čo vytvára dáta → kadiaľ dáta tečú → čo ich spracúva → kam výsledok smeruje.

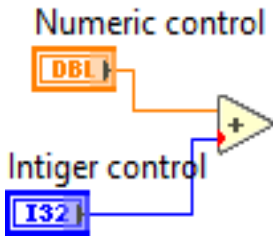
Dátové typy v LabVIEW

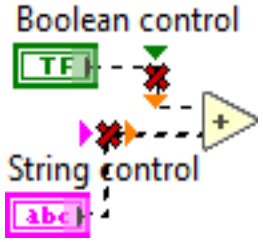
- Dátový typ je v Block Diagram viditeľný – určuje vzhľad terminálov a dátových spojov (wires).

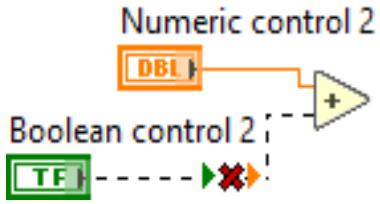


LabVIEW	Význam
Floating point: SGL, DBL, EXT	číslo s pohyblivou rádovou čiarkou
Integer: I8, I16, I32, I64; **Unsigned:** U8, U16, U32, U64	celé číslo so alebo bez znamienka
Boolean	logická hodnota True/False (nie 1/0)
String	textový reťazec

- Terminály možno prepojiť iba vtedy, ak sú ich dátové typy kompatibilné.







Dátový typ určuje, aké hodnoty môže wire prenášať a s ktorými terminálmi ho možno prepojiť.

Zložené dátové typy – Array a Cluster

Viacero hodnôt môžeme v LabVIEW prenášať ako jeden dátový celok.

Array

- Usporiadaná skupina prvkov rovnakého dátového typu (pole), napr.

U = [1,2; 1,4; 1,1; 1,5; 1,3]

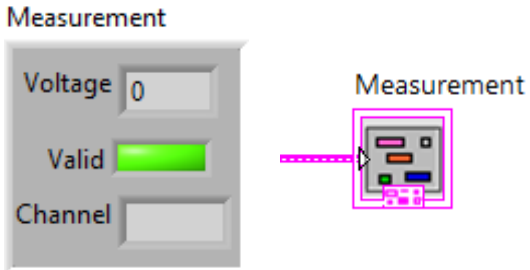


- všetky prvky majú **rovnaký dátový typ**,
- počet prvkov sa môže meniť,
- prvky sú prístupné pomocou **indexu od 0**.

Analógia: C: double u[100], MATLAB: vektor / matica

Cluster

- skupina významovo súvisiacich prvkov, ktoré môžu mať rôzne dátové typy, napr.
- Measurement
- ├── Voltage (DBL)
- ├── Valid (Boolean)
- └── Channel (String)



- prvky môžu mať **rôzne dátove typy**,
- pevne definovaná štruktúra,
- viac súvisiacich údajov sa prenáša **jedným wire**.

Analógia: **C:** struct

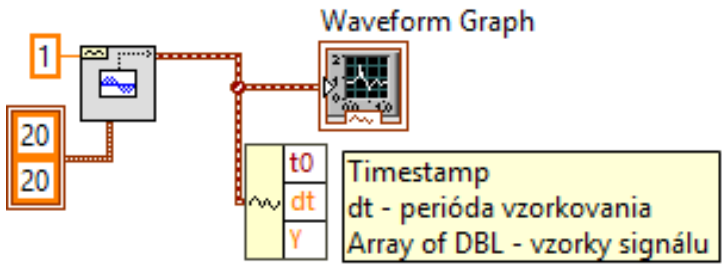
Prečo zoskupovať dáta? → jeden logický dátový celok, menej wires → prehľadnejší diagram

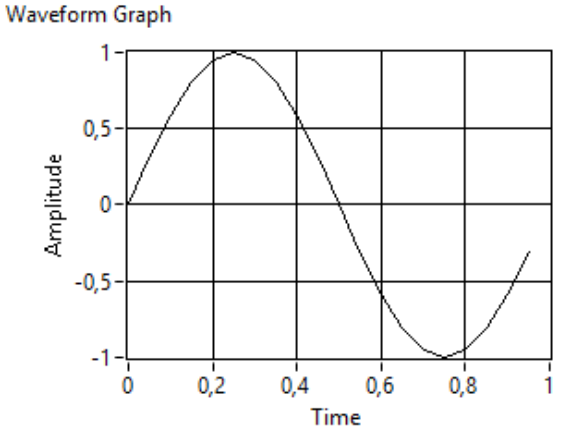
Array združuje viac hodnôt rovnakého typu; Cluster združuje významovo súvisiace hodnoty, ktoré môžu mať rôzne typy.

Dátové typy pre meracie aplikácie

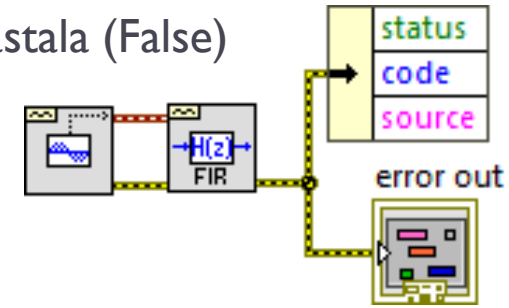
Zložené dátové typy spájajú údaje, ktoré spolu tvoria jeden významový celok.

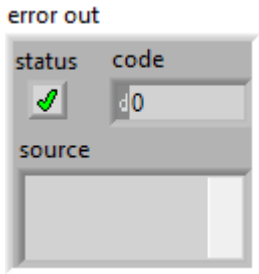
- **Waveform** – vzorkovaný signál
  - Waveform prenáša spolu vzorky signálu aj informáciu potrebnú na určenie ich času.





- **Error Cluster** – informácia o chybe
  - **status** – Boolean chyba nastala (True) / nenastala (False)
  - **code** – Integer identifikácia chyby
  - **source** – String informácia o zdroji chyby





Error Cluster sa prenáša medzi funkciami a SubVI spolu s ostatnými dátami.

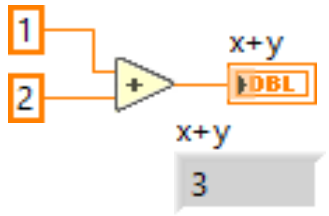
Function → error → SubVI → error → Function

Dátový objekt môže okrem samotnej hodnoty prenášať aj informácie potrebné na jej správnu interpretáciu a spracovanie.

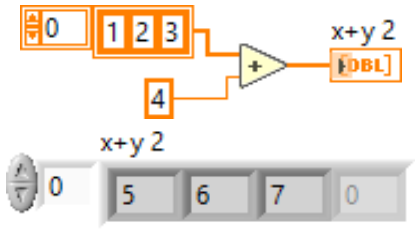
Polymorfné funkcie v LabVIEW

- Potrebujeme inú funkciu sčítania (+) pre DBL, Integer alebo Array?

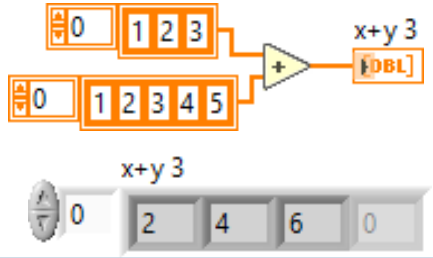
Nie – mnohé funkcie LabVIEW sú polymorfné (polymorphic).



skalár + skalár → skalár



array + skalár → array
skalár sa aplikuje na každý prvok Array



array + array → array
operácia sa vykoná po prvkoch (element-wise)
POZOR: rozmer výsledného poľa je daný najmenšími rozmermi vstupných polí

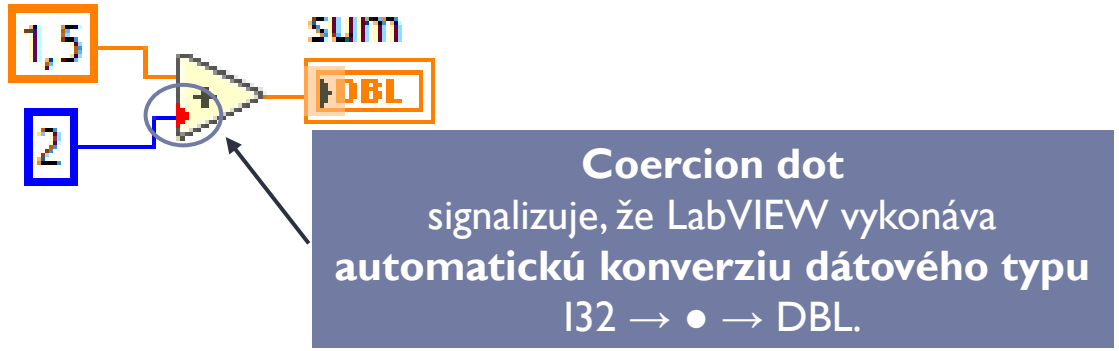
Polymorphic function: jedna funkcia podporuje viacero kompatibilných dátových typov alebo štruktúr dát.
Polymorfná funkcia má definovaných viacero kompatibilných kombinácií typov svojich terminálov

Pozor pri Arrays - Aritmetické operácie nad Arrays sú spravidla vykonávané po prvkoch.
Napríklad pri 2D Arrays: A × B pomocou bežnej funkcie Multiply
→ **násobenie zodpovedajúcich prvkov, nie maticové násobenie (!)**

Polymorfizmus umožňuje použiť tú istú funkciu pre rôzne kompatibilné dátové typy a štruktúry dát.

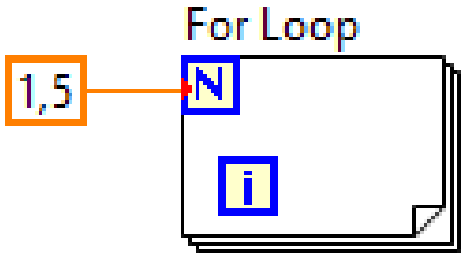
Automatická konverzia dátových typov – coercion

- Čo sa stane, ak prepojíme **kompatibilné**, ale rozdielne dátové typy?



- coercion **nie je chyba**,
- môže spôsobiť **zmenu reprezentácie alebo stratu informácie**,
- neočakávaný coercion dot je dôvod **skontrolovať dátové typy**.

- **Pozor** pri opačnej konverzii, napr. DBL → ● → I32 (počet opakovaní slučky FOR):
  - Konverzia floating point → integer používa zaokrúhlenie na najbližšie celé číslo; presná polovica sa zaokrúhli k najbližšiemu párnemu číslu (round to nearest, ties to even). (5,49 → 5; 5,5 → 6 ale 6,5 → 6; 6,51 →7)
- Ak chceme konverziu dátového typu vyjadriť explicitne, použijeme konverzné funkcie z palety Numeric » Conversion

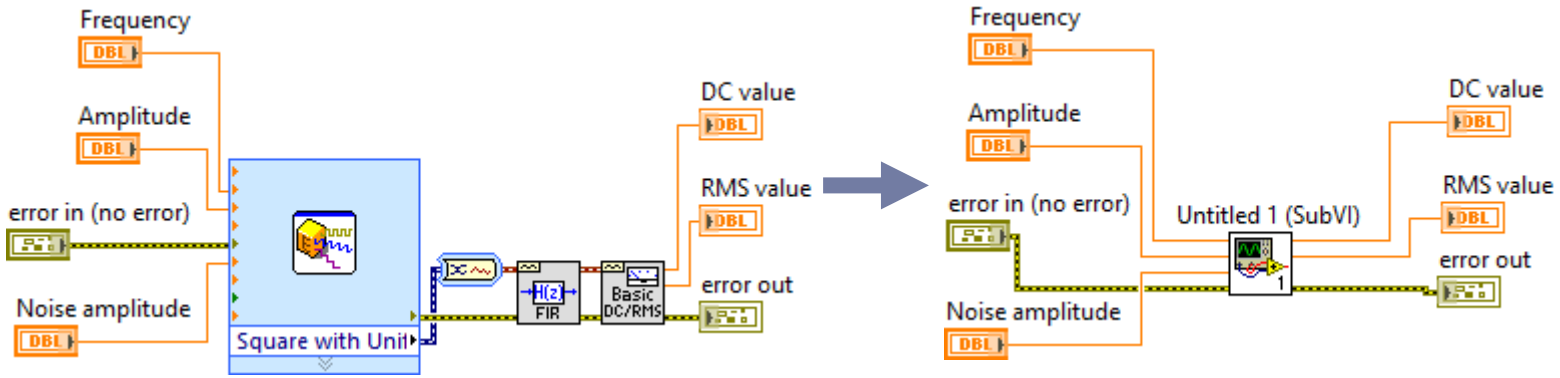


Coercion dot nie je chyba – upozorňuje však na konverziu, ktorej pravidlá a dôsledky musíme poznať.

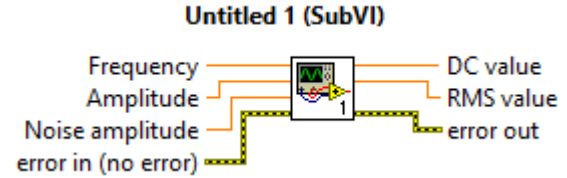


SubVI – modularita programu

VI môže byť samostatný program alebo môže byť použité ako podprogram (SubVI) v inom VI.



VI je samostatná programová jednotka LabVIEW



SubVI = VI použité ako podprogram v Block Diagram iného VI



Connector Pane definuje rozhranie VI – jeho vstupy a výstupy.
Odporúčanie: vstupy vľavo, výstupy vpravo

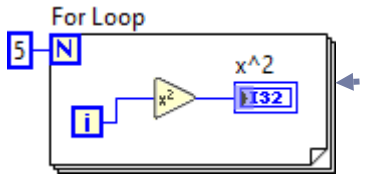
- SubVI plní podobnú úlohu ako funkcia/podprogram v textovom jazyku.
- Prečo SubVI?
  - **Modularita** – rozdelenie programu na logické časti
    Opakované použitie – rovnaké VI možno volať na viacerých miestach
    Prehľadnosť – detail algoritmu sa skryje za zrozumiteľnú ikonu

SubVI podlieha rovnakému modelu dataflow – vykonať sa môže, keď má dostupné všetky požadované vstupné dáta.

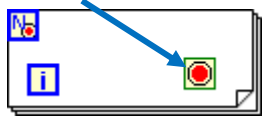
SubVI umožňuje skryť implementáciu za definované dátové rozhranie a vytvárať hierarchický program.

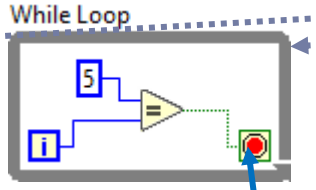
Structures – riadenie vykonávania časti programu

Structures určujú opakovanie alebo podmienené vykonávanie časti Block Diagramu.



For Loop
časť diagramu sa štandardne vykoná **N-krát**,
N – maximálny počet počet iterácií,
i – **iteration terminal**, index od 0.
Analógia: ***C:*** *for (...)*
For je možné aj predčasne ukončiť pridaním **Conditional terminal**

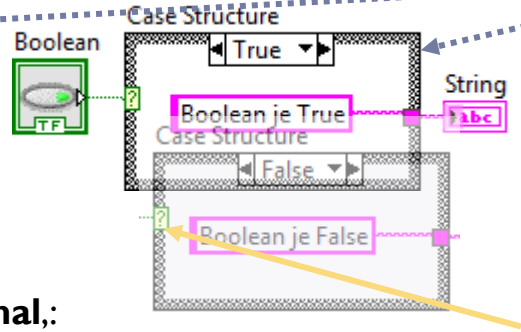




While Loop
opakuje časť diagramu,
ukončenie určuje **conditional terminal**,:
Stop if True
Continue if True



počet iterácií nemusí byť vopred známy.
Analógia: ***C:*** *while (...)*



Case Structure
obsahuje viac alternatívnych diagramov (**cases**),
vykoná sa **iba jeden case**,
výber určuje hodnota na **selector terminal**.
selector môže byť iba **Boolean, Integer, String, Enum a Error cluster.**
Floating-point nie je selector type – pri pripojení sa hodnota konvertuje na Integer.
Analógia: ***C:*** *if / switch*

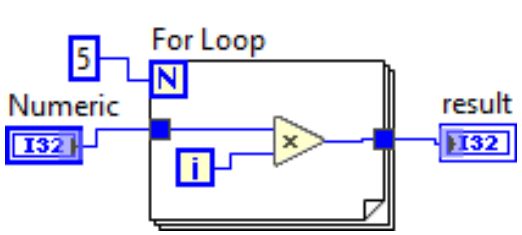
- Structure obsahuje časť Block Diagramu (vnútri rámčeka).
- Structures nemenia princíp dataflow.

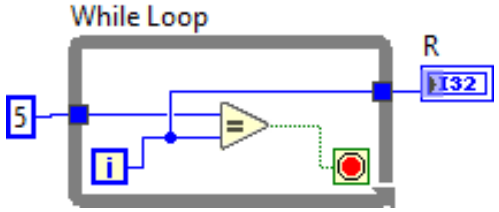
Aj Structure sa môže vykonať až vtedy, keď sú dostupné dáta potrebné na jej vstupoch.

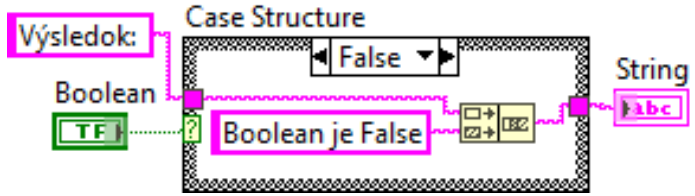


Tunnels – dáta cez hranicu Structure

Ako vstupujú dáta do Structure a ako z nej vystupujú?







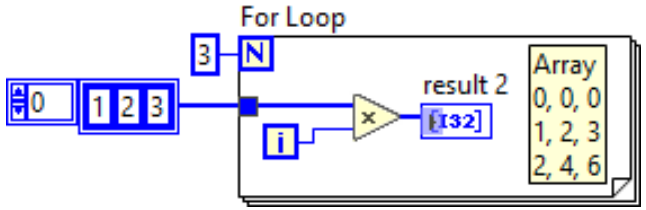
- **Tunel -** miesto, ktorým dáta prechádzajú cez hranicu Structure
  - môže byť **vstupný** alebo **výstupný**
- **Vstupný tunnel**
  - Dáta sa prenesú **pri začatí vykonávania Structure**.
  - **Neskoršia zmena zvonka sa počas jej behu dovnútra neprenesie.**
- **Výstupný tunnel**
  - Dáta sú dostupné mimo Structure **až po jej dokončení**.

Tunnel vytvára dátové rozhranie medzi vnútrom a okolím Structure.
Tunnel nie je „živé spojenie“.

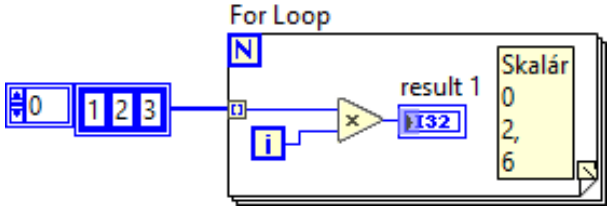


Tunnels – auto-indexing na vstupe

- Čo sa stane, ak na vstup For Loop pripojíme Array?



Jednoduchý tunel = Auto-indexing disabled
V každej iterácií je dostupný celý Array



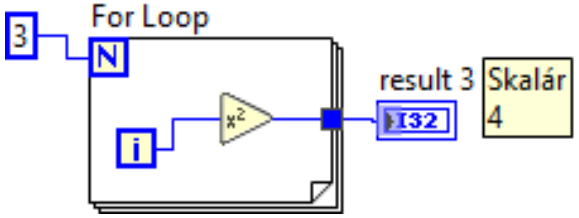
Indexovaný tunel = Auto-indexing enabled (default)
V každej iterácii je dostupný nasledujúci prvok Array

- Auto-indexovaný tunnel môže určiť počet iterácií For Loop
  - Ak počet iterácií určuje N aj auto-indexované vstupy, For Loop sa vykoná podľa najmenšieho dostupného počtu prvkov/iterácií.
  - Ak na vstupe viacero Array s rôznou dĺžkou, počet iterácií = dĺžka najkratšieho Array
- Defaultné nastavenie:
  - **For Loop:** auto-indexing **enabled**
  - **While Loop:** auto-indexing **disabled**

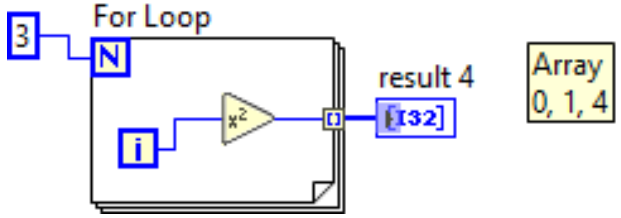
Input auto-indexing: Array sa rozdelí na prvky spracovávané v jednotlivých iteráciách slučky.

Tunnels – auto-indexing na výstupe

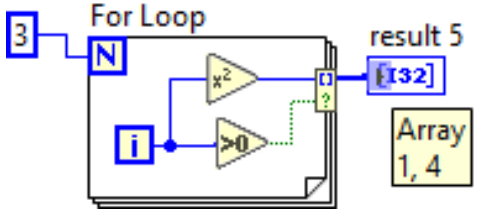
- Čo sa stane s hodnotami vytvorenými v jednotlivých iteráciách slučky?



Jednoduchý tunel:
Auto-indexing disabled
Po dokončení slučky je na výstupe hodnota z poslednej iterácie.



Indexovaný tunel:
Auto-indexing enabled
Po dokončení slučky je na výstupe Array tvorené hodnotami zo všetkých iterácií.



Tunel s podmieneným indexovaním = Conditional indexing
Do výsledného Array sa zaradia iba hodnoty z iterácií, v ktorých je podmienka True.

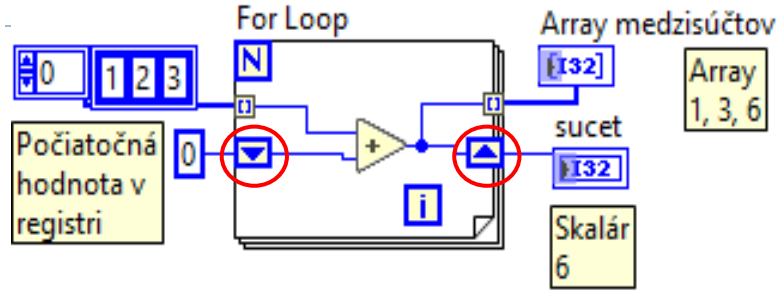
Výstupný tunnel	Výsledok
Indexing disabled	posledná hodnota
Auto-indexing	Array zo všetkých iterácií
Conditional indexing	Array z vybraných iterácií

Výstupný tunnel určuje, či zo slučky získame poslednú hodnotu, všetky hodnoty alebo iba hodnoty spĺňajúce podmienku.

Shift Register – dáta medzi iteráciami

Ako prenesieme výsledok jednej iterácie do nasledujúcej?

Príklad: kumulatívny súčet prvkov Array
(prípadne aj výstup medzisúčtov)



- **Shift register** (posuvný register) prenáša hodnotu z jednej iterácie slučky do nasledujúcej
  - **ľavý terminal** hodnota z predchádzajúcej iterácie
    - **Initialized Shift Register** (v príklade = 0) - počiatočný stav je pri každom vykonaní explicitne daný zvonka.
    - **Uninitialized Shift Register** – nemá zvonka definovanú počiatočnú hodnotu. Jeho hodnota môže pretrvávať medzi opakovanými volaniami VI; pri ďalšom volaní sa použije hodnota zachovaná z predchádzajúceho vykonania.
  - **pravý terminal** hodnota odovzdaná nasledujúcej iterácii

Tunnel vs. Shift Register: **Tunnel:** okolie slučky ↔ slučka vs. **Shift Register:** iterácia i → iterácia i+1

Shift Register uchováva stav slučky – výsledok jednej iterácie môže byť vstupom nasledujúcej.



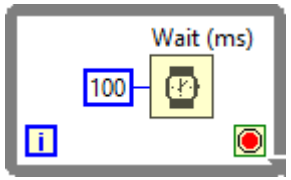
Časovanie slučky – ako často sa vykonáva?

- Ako často sa vykoná ďalšia iterácia While alebo For Loop?
  1. **Ďalšia iterácia začne hneď**, keď to dovolí dataflow. Teda periódu ovplyvňuje:
     - čas vykonania kódu vnútri slučky,
     - plánovanie operačného systému,
     - ostatné bežiace úlohy.

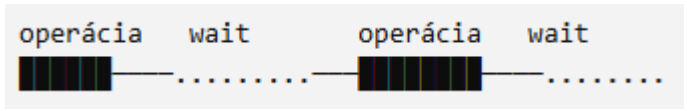


→ **Slučka sama neurčuje periódu vykonávania**

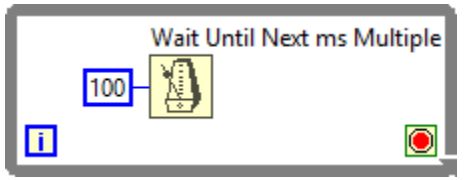
  2. Funkcia **Wait (ms)** – zadrží svoju časť vykonávania najmenej na zadaný čas. V slučke sa používa na obmedzenie rýchlosti opakovania, čas, napr. 100ms



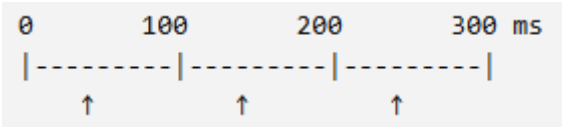
     - Wait ≠ presná perióda slučky, napr. Ak výpočet trvá napr. 30 ms a potom čakáme 100 ms: $T \approx 30+100 \leq 130$ ale ≥ 100 ms (a, samozrejme, plus neurčitosť plánovania OS – všetko beží paralelne).



  3. Funkcia **Wait Until Next ms Multiple** - synchronizuje vykonanie na najbližší násobok zadaného intervalu systémového časovača, čím umožňuje stabilnejšie priemerné opakovanie než jednoduché Wait (ms).



     - **Ani toto ale nie je presné deterministické časovanie!**




```
ideálne:     |----|----|----|----|
skutočné:    |-----|---|------|---|
```

Poznámka: **Jitter** – odchýlka skutočného okamihu vykonania od požadovaného časovania (OS • scheduler • zaťaženie CPU • vykonávaný kód).

Časovanie bežnej programovej slučky je softvérové – závisí od vykonávania programu a operačného systému, preto nie je presne deterministické.

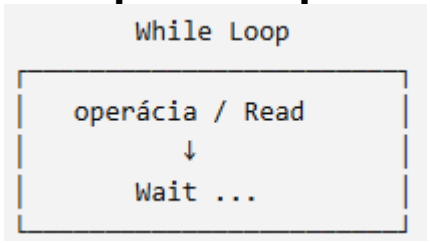
Softvérové vs. hardvérové časovanie

- Čo teda urobíme, ak potrebujeme operáciu vykonávať skutočne v presne definovaných časových okamihoch?
  - Ak sa While Loop opakuje približne každých 10 ms, znamená to, že operácia prebehne presne každých 10 ms?
- NIE – závisí od toho, kto určuje okamih vykonania operácie.

Softvérové časovanie (Software-timed)

- závisí od behu programovej slučky,
- ovplyvňuje ho OS, scheduler a zaťaženie CPU,
- interval medzi operáciami môže kolísať (**jitter**),
- vhodné, ak presné časovanie nie je kritické.

Programová slučka → určuje KEDY program dostupné dáta preberá a spracúva.

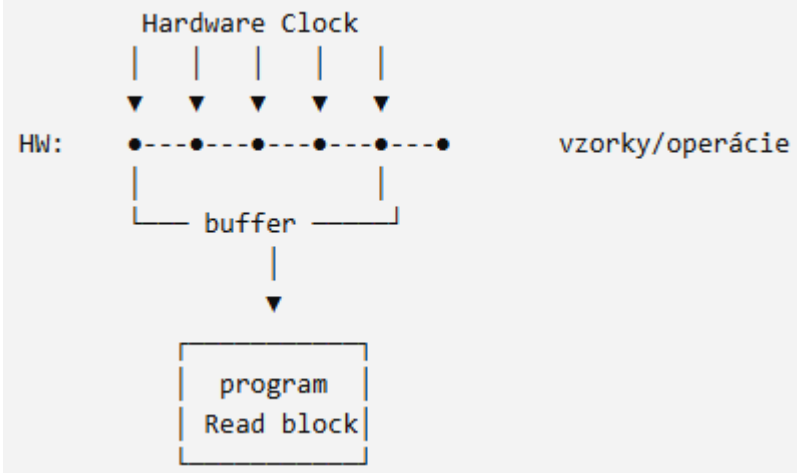


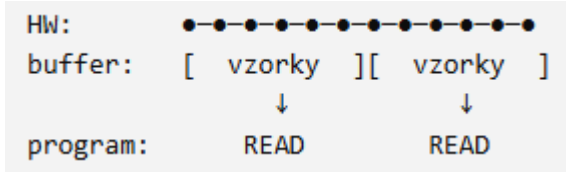


Hardvérové časovanie (Hardware-timed)

- operácie prebiehajú v presne definovaných časových okamihoch,
- časovanie je nezávislé od okamžitého behu aplikačnej slučky,
- dáta sa môžu do programu prenášať **po blokoch cez buffer**.

Hardware clock → určuje KEDY vznikajú jednotlivé vzorky / operácie





Poznámka: **Buffer** dočasne uchováva dáta a oddeľuje presné hardvérové časovanie od nepravidelného behu programu.

Programová slučka časuje vykonávanie programu; hardvérový clock môže nezávisle a presne časovať jednotlivé operácie alebo vzorky.

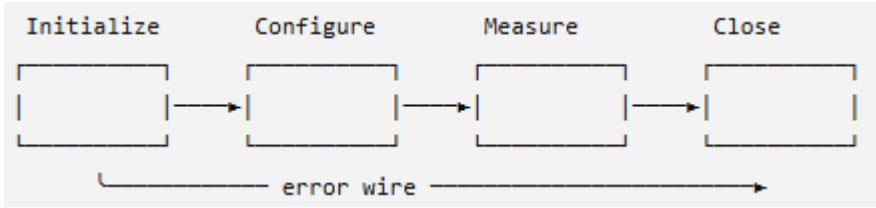
Riadenie poradia vykonávania v dataflow

Ako zabezpečíme, aby sa operácia B vykonala až po dokončení operácie A?

A: Configure B: Measure

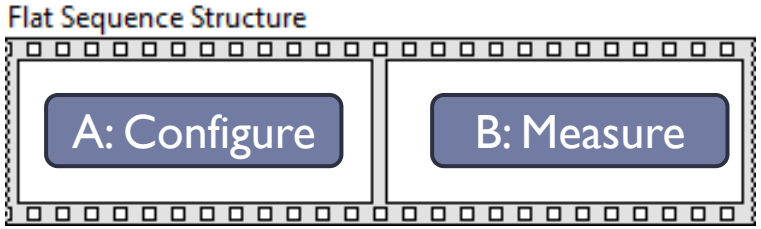
1. Štandardné **odporúčané** riešenie: vytvoriť dátovú závislosť
   - Ak výstup jednej operácie pripojíme na vstup ďalšej, vytvoríme dátovú závislosť (data dependency).
   - Príklad: použite Error wire
     - prenáša informáciu o chybe
     - vytvára dátovú závislosť

A: Configure → dáta/referencia → B: Measure



Wire neurčuje poradie preto, že ide „zľava doprava“. Poradie vzniká preto, že nasledujúca operácia potrebuje dáta vytvorené predchádzajúcou.

2. Ak dátová závislosť nie je k dispozícii → **Sequence Structure** = explicitné určenie poradia
   - Existuje v dvoch grafických podobách: Flat a Stacked
   - Sequence Structure nenahrádza dátové závislosti a nemá sa používať na sekvenovanie celého programu.



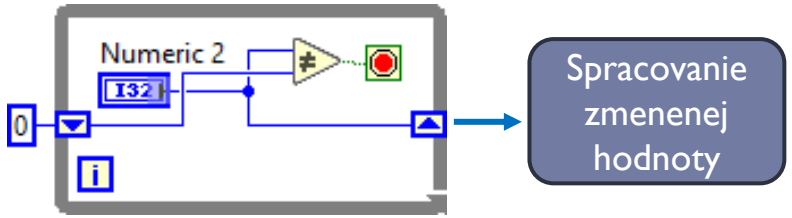
Poradie vykonávania určujeme prednostne dátovou závislosťou; Sequence Structure použijeme iba vtedy, keď požadované poradie nemožno vyjadriť dátovou závislosťou.

Event Structure – reakcia na udalosti

- Ako má program reagovať na aktivitu používateľa?

Napríklad: Používateľ zmení hodnotu Control → program má vykonať určitú činnosť.

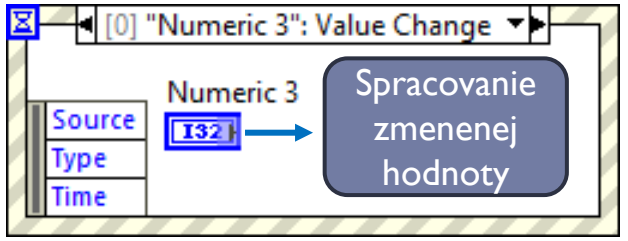
Polling – cyklické zisťovanie stavu



- program **opakovane číta** stav Control (NUMERIC 2),
- musí zisťovať, či nastala zmena,
- slučka pracuje aj vtedy, keď používateľ nič nerobí.

Program aktívne zisťuje, či udalosť nastala. Pri nevhodnej aplikácii alebo časovaní môže spôsobiť zbytočné zaťaženie systému

Event Structure – čakanie na udalosť



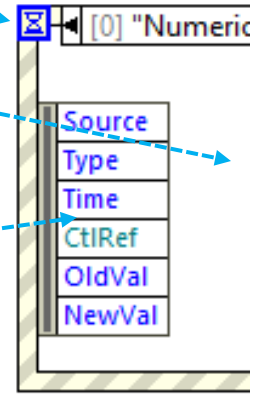
- čaká na definovanú **udalosť (event)** (=zmena Numeric 3)
- príslušný **event case** sa vykoná až po jej vzniku,
- nie je potrebné neustále testovať stav Control.
- nemení princíp dataflow
- koncepčne sa podobá obsluhe udalosti/prerušenia

Polling	Event Structure
program zisťuje stav	program čaká na udalosť
pravidelné čítanie	reakcia pri vzniku eventu
vhodné aj pre periodické činnosti	vhodné pre interakciu používateľa



Event Structure – čakanie na udalosť

- Čo je **event**?
  - napríklad: Value Change, Mouse Down, Key Down, Panel Close?, Timeout, ...
  - **Timeout** – príslušný Event case sa vykoná, ak počas určeného času nenastane (iná) obsluhovaná udalosť.
    - Ak Timeout nie je použitý a nenastane žiadna obsluhovaná udalosť, Event Structure čaká bez časového obmedzenia.
- Čo je **Event case**?
  - časť diagramu obsluhujúca konkrétnu udalosť
- Ten istý Even case môže byť spúšťaný viacerými rôznymi udalosťami podľa nastavenia v Edit Events, napr. zvolený Control a Timeout
  - Control, ktorý generuje event, nemusí byť vo vnútri Event Structure. Ak však v event case potrebujeme jeho aktuálnu hodnotu, použijeme terminal vo vnútri case alebo NewVal z Event Data Node.
- V interaktívnych aplikáciách sa Event Structure typicky umiestňuje do While Loop, aby program mohol opakovane čakať na ďalšie udalosti.
- **Event Data Node** poskytuje údaje o vzniknutej udalosti (NewVal, CtlRef, Time, ...).



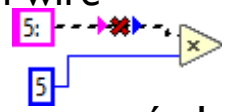
Event Structure umožňuje vykonať časť programu ako reakciu na vznik definovanej udalosti bez neustáleho zisťovania stavu.

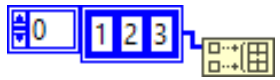


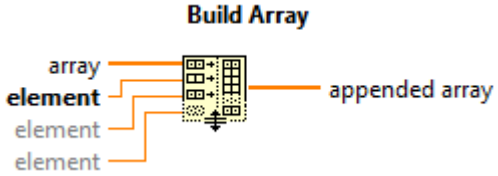
Ladenie programu – Debugging

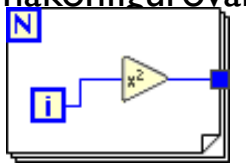
- Čo robiť, ak VI nemožno spustiť alebo program nepracuje podľa očakávania?

1. VI nemožno spustiť -  - Block Diagram obsahuje chybu, ktorá bráni spusteniu VI.
   Error List - zobrazí chyby brániace spusteniu a umožní lokalizovať problém v Block Diagram. Typické príklady

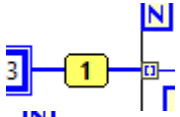
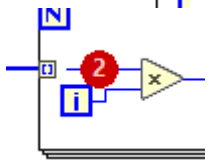
broken wire 

nepripojený required input 



neúplná/nesprávne nakonfigurovaná Structure 

2. VI sa spustí, ale nepracuje správne
   -  **Execution Highlighting** vizualizuje priebeh vykonávania a tok dát a **umožňuje pozorovať dataflow programu**.
   -  **Single-Stepping** – riadené vykonávanie po jednotlivých krokoch:

     Step Into – vstúpi do SubVI, **Step Over** – vykoná SubVI bez vstupu do jeho Block Diagram, **Step Out** – dokončí aktuálne VI a vráti sa do volajúceho VI.
   -  **Probe** (obdoba Watch v C) zobrazuje hodnotu dát na vybranom wire počas vykonávania program. Ak výsledok nie je správny, sledujme, kde sa dáta prvýkrát odchýlia od očakávanej hodnoty.
   -  **Breakpoint** pozastaví vykonávanie na zvolenom mieste. Dá sa výhodne kombinovať s Probe.

3. Runtime error - program sa vykonáva, ale počas behu vznikne chyba, napr.:

   File I/O → komunikácia → neplatné dáta/parameter → HW

Pri ladení dataflow programu sledujeme, kedy sa jednotlivé časti vykonávajú a aké dáta medzi nimi tečú.

Zobrazovanie dát – Indicators a Graphs

- Ako zobraziť dáta podľa ich štruktúry a spôsobu, akým vznikajú?

1. **Numeric Indicator = jedna hodnota (**skalár), Napr. výsledok výpočtu y = 4,73
2. **Waveform Chart** - hodnoty prichádzajú (vznikajú) postupne: $y_0 \rightarrow y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow \ldots$ a postupne sa dokreslujú do grafu (obrázok). Chart teda **postupne pridáva** nové hodnoty a **uchováva históriu**.
3. **Waveform Graph** - dáta sú k dispozícii ako blok (napr. Array alebo Waveform): $[y_0,y_1,y_2,\ldots,y_N]$. Graph pri aktualizácii nahradí celé zobrazenie novým dodaným dátovým blokom.
4. **XY Graph** - explicitná závislosť Y od X. Graph pri aktualizácii zobrazuje body definované explicitnými dvojicami X–Y, vo forme Cluster {X Array, Y Array}
   Príklady: y=f(x)parametrická krivka, nerovnomerne rozmiestnené hodnoty X, …
5. Ďalšie špeciálne možnosti ako Intensity, Digital waveform, Mixed waveform, 3D, …

	Waveform Chart	Waveform Graph
Vstup	hodnoty postupne	blok dát
História	vytvára a uchováva Chart	dodáva ju program
Aktualizácia	pridanie nových dát	zobrazenie dodaného datasetu

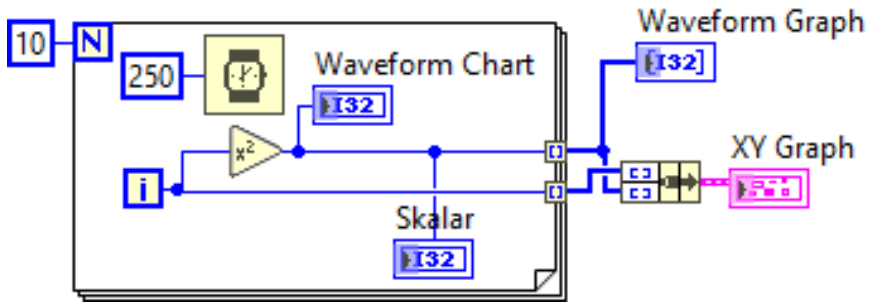


Chart vytvára históriu z postupne prichádzajúcich dát; Graph zobrazuje dátový blok; XY Graph zobrazuje explicitné dvojice X–Y.

Dátové formáty a ich zobrazenie

- Graph a Chart interpretujú vstupné dáta podľa ich dátového typu a štruktúry.

Waveform Graph

- Jednorozmerne pole: 1D Array Y = [2, 4, 3, 6, 5] → X je implicitne dané indexom 0, 1, 2, ...
- Waveform → os X je určená hodnotami t_0 a dt
- 2D Array – viac priebehov
- Array of Waveforms – viac priebehov

XY Graph

Jeden XY priebeh – Bundle X a Y, napr. Cluster {X Array, Y Array}
Viac XY priebehov - Array of XY data

Waveform Chart

Skalár pridá jednu hodnotu
Array alebo Waveform pridá blok
Blok nových nasledujúcich hodnôt

Poznámka: Chart môže svoju históriu zobrazovať troma spôsobmi::
Chart môže dopĺňat nové hodnoty troma spôsobmi:
***Strip chart** – nové dáta postupne prepisujú staré zľava doprava.*
***Scope chart** – po zaplnení okna grafu sa obrázok zmaže a nové hodnoty sa dopĺňajú od začiatku vľavo.*
***Sweep chart** – nakreslený graf sa postupne zľava prepisuje novými pridávanými hodnotami.*

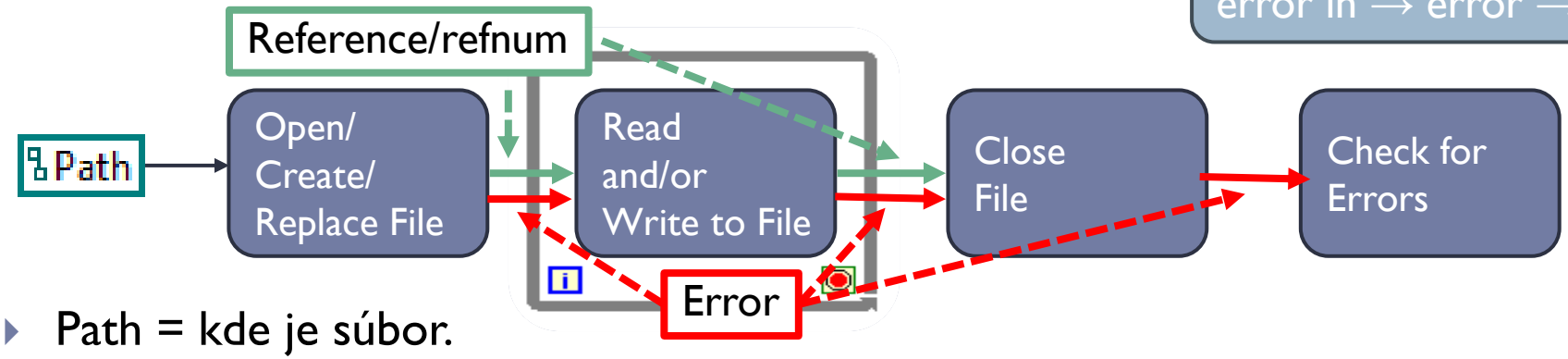
Pri pripájaní dát ku Graph alebo Chart musíme poznať nielen dátový typ, ale aj očakávanú štruktúru dát.



Práca so súbormi – File I/O

Ako program číta alebo zapisuje dáta do súboru?

Path → Open/Create → Read / Write → Close
error in → error → error → error out



- Path = kde je súbor.
  - identifikuje umiestnenie súboru alebo adresára, C:\Windows\System32
  - v LabVIEW má vlastný dátový typ **Path** (nie je to String)
- Reference / refnum identifikuje otvorený súbor, s ktorým pracujú ďalšie File I/O funkcie.

LabVIEW ponúka:

- **high-level File I/O** – viac krokov realizuje jedna funkcia,
- **low-level File I/O** – explicitné Open → Read/Write → Close.

Typický životný cyklus práce so súborom je: Open/Create → Read/Write → Close.

Formát dát v súbore

DBL 1.25
- Text → "1.25"
- Binary → binárna reprezentácia hodnoty DBL

- Ako chceme dáta v súbore reprezentovať?
  - **File I/O** → *ako so súborom pracujeme* (Open → Read/Write → Close)
  - **File format** → *ako sú dáta v súbore uložené a interpretované*

Text
- ✓ dáta sú uložené ako textové znaky napr. ASCII, UTF-8, …
- ✓ človekom čitateľný,
- ✓ jednoduchá výmena medzi programami,
- ✓ čísla sa musia konvertovať **number ↔ string**,
- ✓ spravidla väčší objem dát.

Príklad: .csv, .txt, .m, .cpp, ...

Binary
- ✓ dáta sú uložené v **binárnej reprezentácii**,
- ✓ kompaktné a efektívne,
- ✓ bez konverzie číslo ↔ text,
- ✓ na správne načítanie/zápis musíme poznať **dátový typ a štruktúru dát**.

Pri binárnom súbore nestačí poznať jednotlivé bajty – musíme vedieť, ako ich interpretovať.

TDMS
- ✓ štruktúrovaný súborový formát,
- ✓ **dáta + metadata**,
- ✓ hierarchia **File → Group → Channel**,
- ✓ navrhnutý pre technické/experimentálne dáta,
- ✓ podporuje efektívny zápis väčších objemov dát.
- ✓ možno načítať/analyzovať aj externými nástrojmi (napr. Excel)

Formát súboru určuje, ako sa hodnoty a ich štruktúra ukladajú a ako ich musí program pri čítaní interpretovať.



Spracovanie chýb – Error Handling

- Čo má program urobiť, ak počas vykonávania vznikne chyba?

Detect → Propagate → Handle → Cleanup

Detect
- funkcia/SubVI signalizuje chybu cez error out

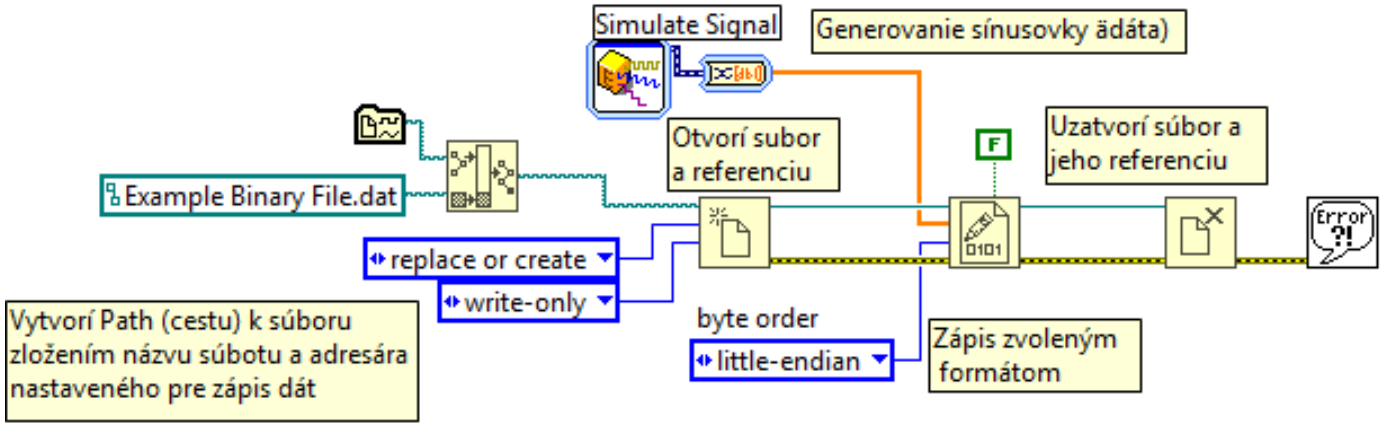
Propagate
- error wire prenáša Error Cluster ďalej

Handle
- program rozhodne, ako na chybu reagovať

Cleanup
- uvoľní použité zdroje aj v prípade chyby

- **Nie každá chyba musí znamenať ukončenie programu** – reakcia závisí od aplikácie.
- **<u>Aj pri chybe</u> treba korektne uvoľniť zdroje** – napr. zatvoriť otvorený súbor.

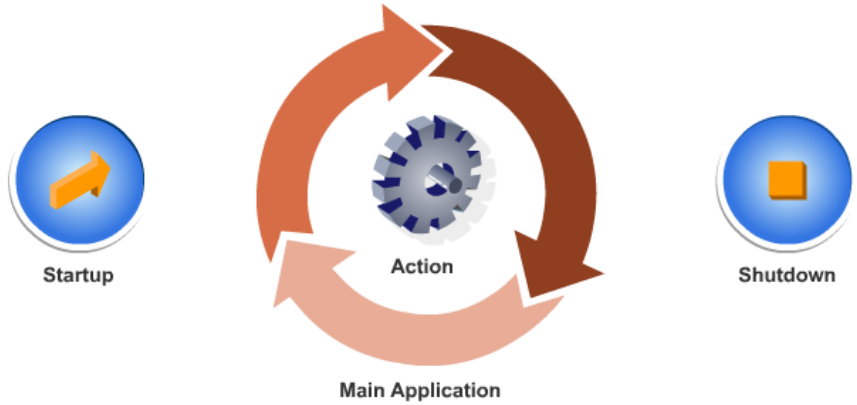
Funkcia **Open/Create/Replace File vytvorí nový súbor. Write to Binary File zapíše dáta (sínusovka) formátom** little-endian (LSB DBL je na prvej pozicii v poradí zápisu do súboru). **Close File zatvorí súbor a jeho referenciu.**



Robustný program chybu zistí, prenesie informáciu o nej, primerane zareaguje a korektne uvoľní použité zdroje.

Od jednoduchého VI k aplikácii

- Ako organizovať program, ktorý už nerobí iba jednu operáciu?
- Typické požiadavky:
  - inicializovať potrebné zdroje,
  - opakovane vykonávať hlavnú činnosť,
  - reagovať na používateľa alebo udalosti,
  - spracovať chyby,
  - korektne ukončiť činnosť a uvoľniť zdroje.



- **Startup** = inicializácia – príprava a inicializácia zdrojov, nastavenie počiatočného stavu
- **Main Application** = run - opakovaná hlavná činnosť + reakcia na udalosti
  - využíva už známe Structures, SubVI, stavové dáta a error handling
- **Shutdown** = ukončenie - korektné ukončenie aktivít a uvoľnenie zdrojov
- **Inicializáciu a ukončenie vykonávame typicky raz; hlavná činnosť aplikácie prebieha opakovane.**

Pri väčšej aplikácii už nestačí vedieť, čo jednotlivé časti robia – musíme určiť aj kedy a v akom stave sa majú vykonať.



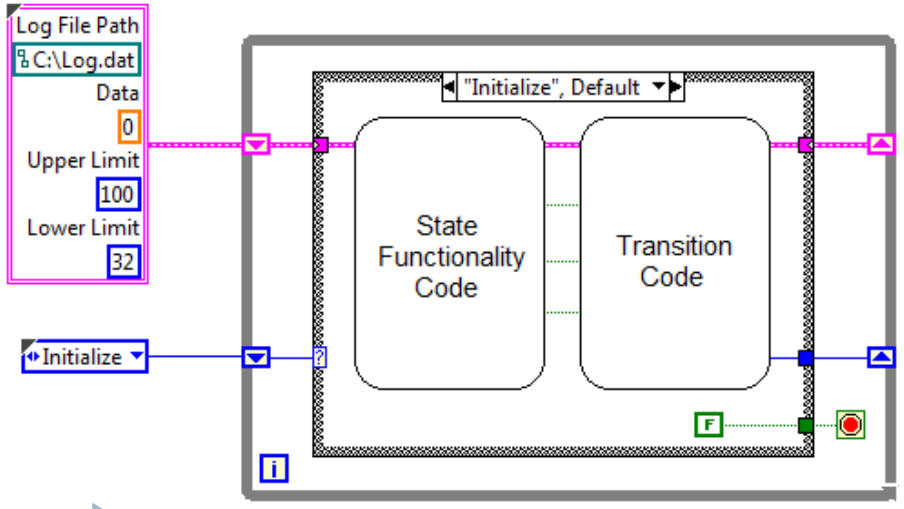
State Machine – stavy a prechody

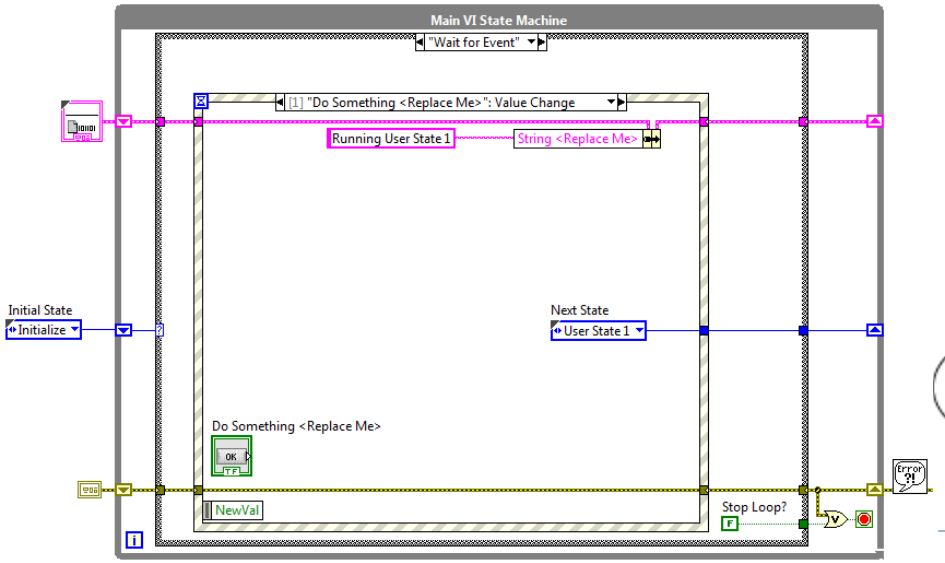
- Ako určíme, čo má aplikácia vykonať teraz a čo bude nasledovať?
- Jedným z typických riešení je State Machine
  - **State (stav)** → určuje, **čo sa práve vykoná**
  - **Transition (prechod)** → určuje, **ktorý stav bude nasledovať**
  - **Transition condition** → udalosť, výsledok operácie, chyba, používateľský vstup.
- Ako sa to dá v LabVIEW realizovať?

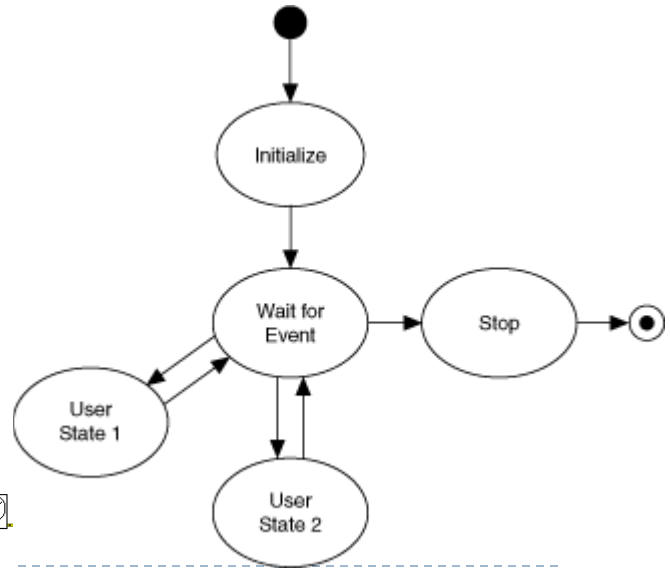










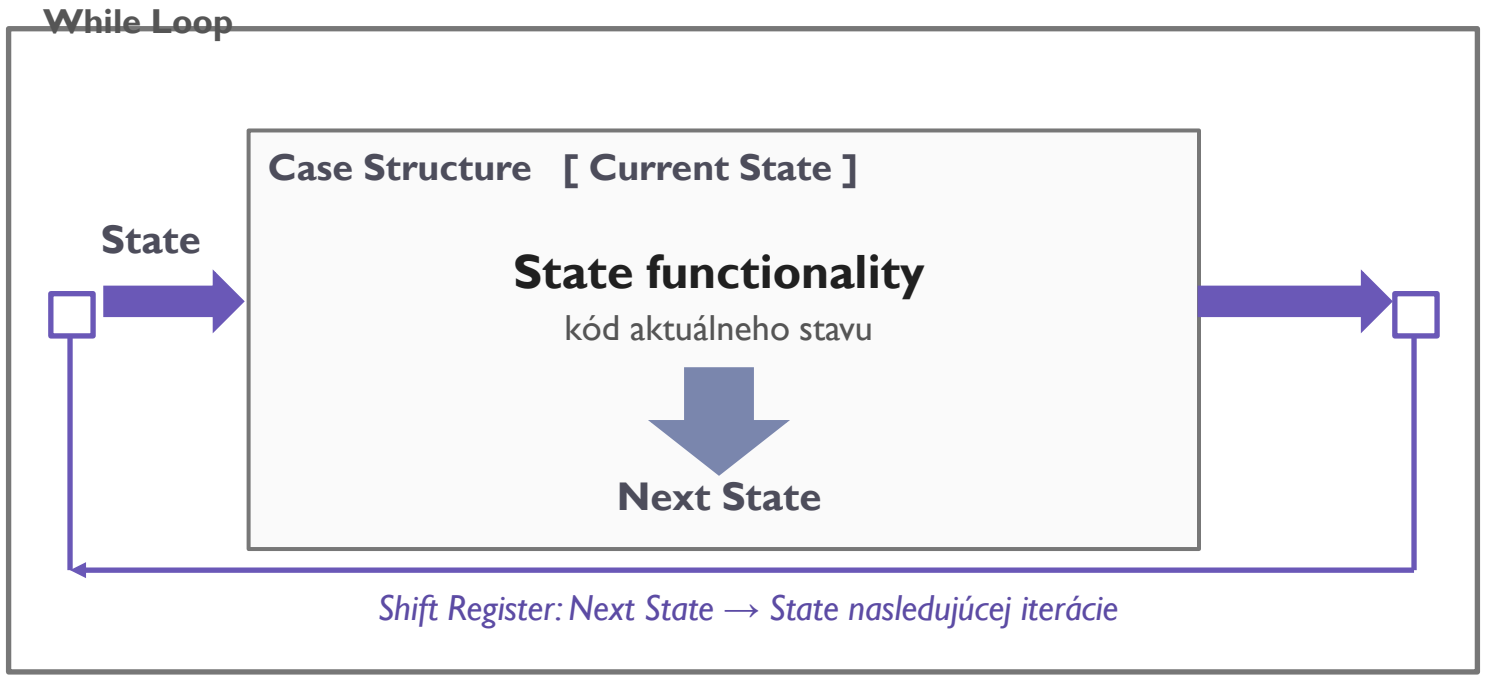


State Machine v LabVIEW

While Loop
opakuje vykonávanie stavov

Case Structure
podľa State vyberie kód aktuálneho stavu

Shift Register
prenesie Next State do nasledujúcej iterácie



Príklad priebehu

iterácia	State	Next State
0	Initialize	Idle
1	Idle	Process
2	Process	Idle
3	Idle	Shutdown

State je vhodné reprezentovať napr. typom Enum:
Initialize | Idle | Process | Error | Shutdown

State Machine = While Loop opakuje → Case Structure vyberá stav → Shift Register prenáša nasledujúci stav.

Zdieľanie dát – Local a Global Variables

- Ako sprístupniť hodnotu na inom mieste programu bez priameho wire?

Local Variable
- Zdieľaná v rámci jedného VI
- viazaná na konkrétny Control alebo Indicator daného VI,
- umožňuje jeho hodnotu **čítať alebo zapisovať** z iného miesta Block Diagramu (typické využitie),
- použiteľná napr. medzi paralelnými časťami **toho istého VI**.

Control X ↔ Local Variable X

Global Variable
- hodnota môže byť zdieľaná **medzi viacerými VI**,
- jednotlivé VI môžu hodnotu čítať alebo zapisovať,
- implementovaná pomocou **Global Variable VI**.



- Local aj Global Variable obchádzajú explicitný dátový tok cez wires.
- POZOR: Local/Global Variable ≠ mechanizmus spoľahlivého prenosu postupnosti dát medzi paralelnými úlohami (race condition – výsledok závisí od časového poradia súbežných operácií– môže viesť k nesprávnym alebo nepredvídateľným výsledkom).

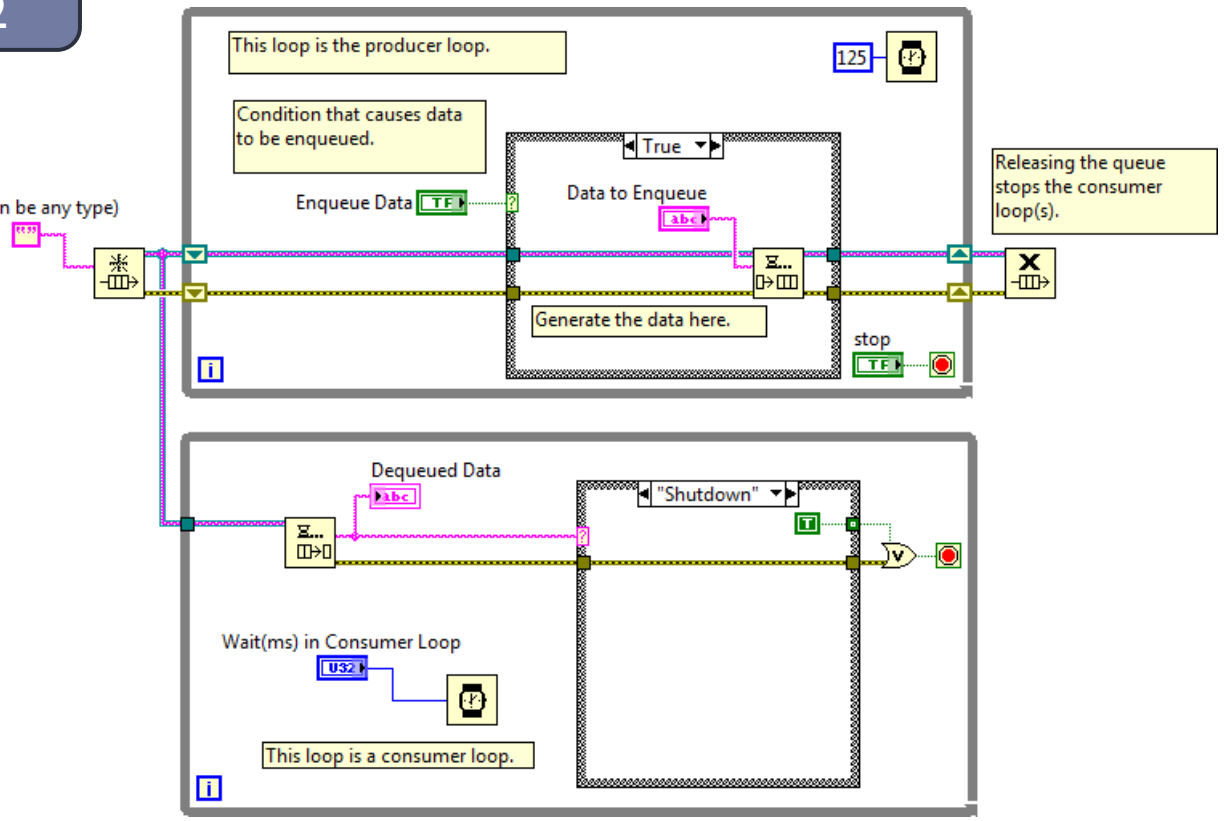
Premenné umožňujú zdieľať aktuálnu hodnotu, ale obchádzajú explicitné dátové závislosti dataflow.

Producer/Consumer - Queue



- **Producer** → vytvára položky a vkladá ich do Queue (Enqueue)
- **Queue – FIFO** → uchováva položky v poradí vloženia
- **Consumer** → odoberá položky (Dequeue) a spracúva ich vlastným tempom

- Queue oddeľuje časovanie Producer a Consumer.
- Ak Queue neobsahuje dáta, Dequeue môže čakať na novú položku – Consumer nemusí cyklicky zisťovať ich dostupnosť.



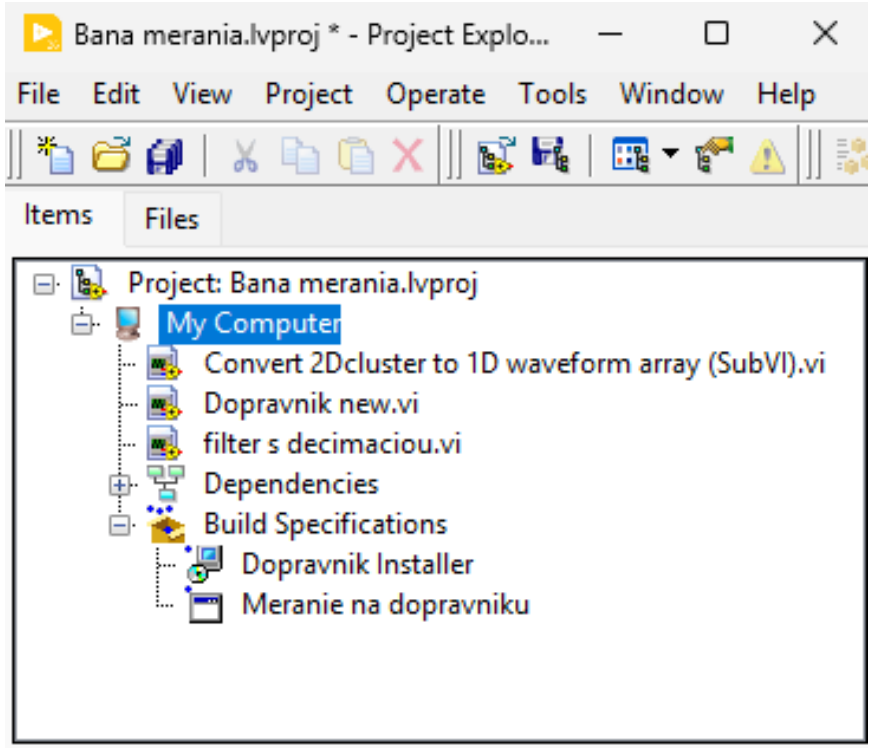
Poducer/Consumer používa Queue na spoľahlivý asynchrónny prenos postupnosti dát medzi paralelnými slučkami.

LabVIEW Project – organizácia aplikácie

- Ako organizovať aplikáciu, ktorá už pozostáva z viacerých VI a ďalších súborov?

Project (.lvproj)
organizuje súbory a komponenty aplikácie,
potrebný najmä pre rozsiahlejšie aplikácie
Typicky obsahuje najmä:

- hlavné VI (Main.vi),
- SubVI,
- Global Variable VI,
- Používateľské typy (.ctl, typedef).
- Dependencies - automaticky identifikované závislosti.,
- prípadne dátové/súborové zdroje,
- Build Specifications pre tvorbu .exe, .dll, inštalačných súborov, source distribution, ...
- ...



Project nie je samotný program – je kontajnerom, ktorý organizuje jeho súčasti a závislosti.



Od LabVIEW k meraciemu systému

Ako teraz našu aplikáciu prepojíme so skutočným meracím prístrojom?

dataflow - UI - spracovanie - súbory – architektúra → ? → merací hardvér